

Trabajo Práctico Integrador

Gestión de Datos de Países en Python

Filtros, Ordenamientos y Estadísticas

Materia:	Programación 1
Institución:	UTN — Tecnicatura Universitaria en Programación
Modalidad:	A Distancia
Integrantes:	Alumno 1 — Alumno 2
Fecha:	Junio 2025

ÍNDICE

1. Marco Teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	4
1.4 Condicionales	4
1.5 Ordenamientos	5
1.6 Estadísticas Básicas	5
1.7 Archivos CSV	6
2. Decisiones Técnicas y Arquitectura	7
2.1 Diagrama de flujo	7
2.2 Estructura del sistema	8
3. Bibliografía / Webgrafía	9

1. MARCO TEÓRICO

1.1 Listas

Una **lista** en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos de cualquier tipo bajo un mismo nombre de variable. Se definen con corchetes `[ ]` y sus elementos se acceden mediante índices numéricos comenzando desde 0. En este proyecto, la variable global **países** es una lista de diccionarios que actúa como la base de datos en memoria del sistema.

```
# Lista vacía que almacenará todos los países
países = []

# Agregar un elemento
países.append({"nombre": "Argentina", "poblacion": 45376763})

# Acceder al primer elemento
print(países[0]["nombre"]) # Argentina

# Longitud
print(len(países)) # 1
```

Las listas admiten operaciones como **append()** para agregar, iteración con **for**, filtrado con comprensiones y ordenamiento con **sorted()**. Son fundamentales para almacenar colecciones dinámicas de datos.

1.2 Diccionarios

Un **diccionario** es una estructura de datos que almacena pares **clave: valor**. A diferencia de las listas, el acceso a los elementos se realiza por clave (string o número), no por posición. En este proyecto, cada país es representado como un diccionario con las claves *nombre*, *poblacion*, *superficie* y *continente*.

```
# Cada país es un diccionario
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "America"
}

# Acceder a un campo
print(pais["nombre"]) # Argentina

# Modificar un campo
```

```

pais["poblacion"] = 46000000

# Iterar claves y valores
for clave, valor in pais.items():
    print(f"{clave}: {valor}")

```

Los diccionarios son ideales para modelar entidades con múltiples atributos heterogéneos. También se utilizan en este sistema para contar países por continente (*continentes = {}*), donde la clave es el nombre del continente y el valor es el conteo.

1.3 Funciones

Una **función** es un bloque de código reutilizable que realiza una tarea específica. Se definen con la palabra clave **def** y pueden recibir parámetros y retornar valores. El principio de **una función = una responsabilidad** garantiza código modular, legible y fácil de mantener.

```

def validar_numero(mensaje):
    """Solicita un número entero positivo al usuario."""
    while True:
        try:
            numero = int(input(mensaje))
            if numero < 0:
                print("Error: debe ser positivo.")
            else:
                return numero
        except ValueError:
            print("Error: ingrese un número válido.")

```

En este proyecto se utilizan funciones para cada funcionalidad: *cargar_paises()*, *agregar_pais()*, *buscar_pais()*, *filtrar_continente()*, *ordenar_paises()*, *mostrar_estadisticas()*, etc. Cada función tiene una única responsabilidad, lo que facilita la depuración y el testing.

1.4 Condicionales

Las **estructuras condicionales** permiten ejecutar diferentes bloques de código según si una condición es verdadera o falsa. Python utiliza **if / elif / else**. En este sistema se emplean extensamente para validar entradas, controlar el menú principal, verificar si un país existe antes de actualizarlo y filtrar resultados.

```
# Control del menú principal
if opcion == "1":
    agregar_pais()
elif opcion == "2":
    actualizar_pais()
elif opcion == "3":
    buscar_pais()
elif opcion == "0":
    print("Programa finalizado.")
    break
else:
    print("Opción inválida.")

# Validación en bucle
if texto == "":
    print("Error: no puede estar vacío.")
elif not texto.replace(" ", "").isalpha():
    print("Error: solo se permiten letras.")
else:
    return texto
```

Las condicionales también se combinan con bucles **while True** en las funciones de validación, garantizando que el programa no avance hasta recibir datos correctos.

1.5 Ordenamientos

El **ordenamiento** consiste en reorganizar los elementos de una colección según un criterio determinado. Python provee la función **sorted()** que retorna una nueva lista ordenada, y el método **.sort()** que ordena en el lugar. Ambos aceptan el parámetro **key** (función que indica qué campo usar para comparar) y **reverse** (True para descendente).

```
# Ordenar por nombre ascendente
ordenados = sorted(paises, key=lambda x: x["nombre"])

# Ordenar por población descendente
ordenados = sorted(paises,
```

```
key=lambda x: x["poblacion"],  
reverse=True)
```

```
# Ordenar por superficie ascendente
```

```
ordenados = sorted(paises, key=lambda x: x["superficie"])
```

Se utiliza una **función lambda** como argumento de *key*. Una lambda es una función anónima de una sola expresión. En el ejemplo, *lambda x: x["poblacion"]* le indica a *sorted()* que compare los diccionarios por el campo *poblacion*. Internamente Python aplica el algoritmo **Timsort**, de complejidad $O(n \log n)$.

1.6 Estadísticas Básicas

Las **estadísticas básicas** son cálculos que resumen las características de un conjunto de datos. En este proyecto se calculan:

- **Máximo y mínimo:** país con mayor y menor población, usando *max()* y *min()* con *key*.
- **Promedio (media aritmética):** suma total dividida por la cantidad de elementos.
- **Conteo por categoría:** cantidad de países agrupados por continente.

```
# País con mayor población
```

```
mayor = max(paises, key=lambda x: x["poblacion"])
```

```
# País con menor población
```

```
menor = min(paises, key=lambda x: x["poblacion"])
```

```
# Promedio de población
```

```
promedio_pob = sum(p["poblacion"] for p in paises) / len(paises)
```

```
# Promedio de superficie
```

```
promedio_sup = sum(p["superficie"] for p in paises) / len(paises)
```

```
# Cantidad por continente (diccionario contador)
```

```
continentes = {}
```

```
for pais in paises:
```

```
    c = pais["continente"]
```

```
    continentes[c] = continentes.get(c, 0) + 1
```

1.7 Archivos CSV

Un archivo **CSV** (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos están separados por comas. Es ampliamente utilizado para intercambiar datos entre sistemas. En este proyecto se usa como base de datos persistente: se carga al iniciar el programa y se actualiza cada vez que se agrega o modifica un país.

Estructura del archivo paises.csv:

```
nombre,poblacion,superficie,continente
Argentina,45376763,2780400,America
Brasil,213993437,8515767,America
Japon,125800000,377975,Asia
```

Lectura del CSV (sin módulo csv):

```
with open("paises.csv", "r", encoding="utf-8") as archivo:
    next(archivo) # Saltar encabezado
    for linea in archivo:
        datos = linea.strip().split(",") # Separar por coma
        if len(datos) != 4:
            continue # Ignorar líneas malformadas
        pais = {
            "nombre": datos[0],
            "poblacion": int(datos[1]),
            "superficie": int(datos[2]),
            "continente": datos[3]
        }
        paises.append(pais)
```

Escritura al agregar un país (modo append):

```
with open("paises.csv", "a", encoding="utf-8") as archivo:
    archivo.write(
        f"\n{pais['nombre']},{pais['poblacion']},"
        f"{pais['superficie']},{pais['continente']}"
    )
```

Se utiliza **encoding='utf-8'** para soportar caracteres especiales (tildes, ñ). El modo **'r'** abre para lectura, **'a'** para agregar al final y **'w'** para sobrescribir completamente (usado al actualizar un país).

2. DECISIONES TÉCNICAS Y ARQUITECTURA

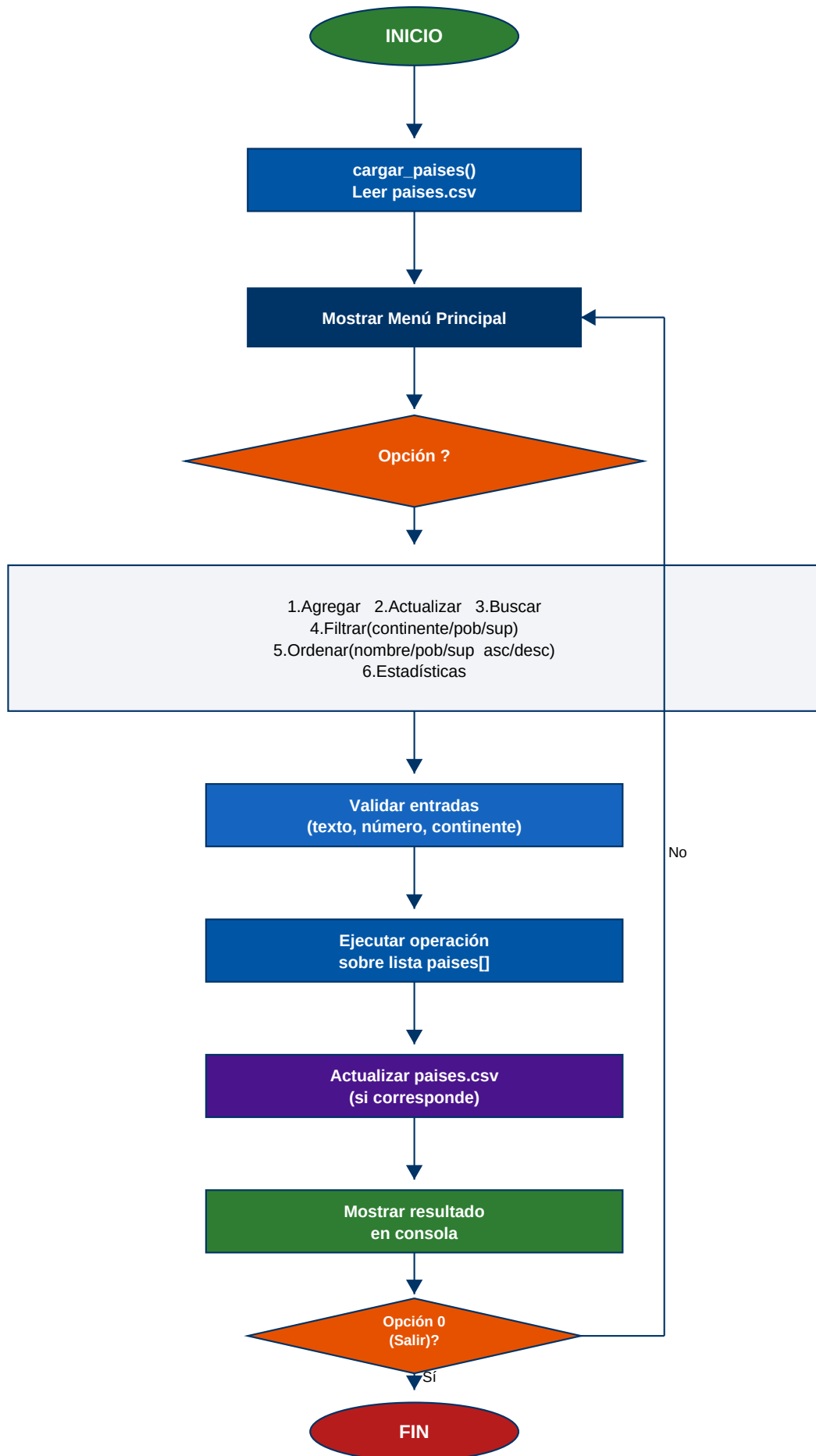
2.1 Estructura del Sistema

El sistema está organizado en módulos funcionales dentro de un único archivo Python. Cada responsabilidad tiene su propia función, siguiendo el principio de **Single Responsibility**. La capa de datos usa una lista global de diccionarios sincronizada con el archivo CSV.

Módulo	Funciones principales	Descripción
Carga/Guardado	cargar_paises() guardar_pais_csv() guardar_todos_csv()	Persistencia en archivo CSV
Validaciones	validar_texto() validar_numero() validar_continente()	Control de entradas del usuario
CRUD	agregar_pais() actualizar_pais() buscar_pais()	Operaciones sobre los datos
Filtros	filtrar_continente() filtrar_poblacion() filtrar_superficie()	Subconjuntos del dataset
Ordenamiento	ordenar_paises()	Sorted con key y reverse
Estadísticas	mostrar_estadisticas()	Max, min, promedio, conteo
Menú	mostrar_menu() menu_filtros()	Interfaz de usuario en consola

2.2 Diagrama de Flujo — Operaciones Principales

A continuación se presenta el diagrama de flujo del programa, desde el inicio hasta la salida, pasando por cada opción del menú principal.



Leyenda del diagrama

Forma	Significado
Óvalo verde/rojo	Inicio / Fin del programa
Rectángulo azul	Proceso / Operación
Rectángulo gris	Detalle de suboperaciones
Rombo naranja	Decisión (bifurcación del flujo)

3. BIBLIOGRAFÍA / WEBGRAFÍA

- [1] Python Software Foundation. (2024). *Python 3 Documentation*. <https://docs.python.org/3/>
- [2] Python Software Foundation. (2024). *Built-in Functions — sorted(), max(), min(), sum()*. <https://docs.python.org/3/library/functions.html>
- [3] Python Software Foundation. (2024). *Data Structures — Lists and Dictionaries*. <https://docs.python.org/3/tutorial/datastructures.html>
- [4] Python Software Foundation. (2024). *Reading and Writing Files — open(), CSV*. <https://docs.python.org/3/tutorial/inputoutput.html>
- [5] Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
- [6] Van Rossum, G. & Drake, F. L. (2009). *The Python Language Reference*. Python Software Foundation.
- [7] Real Python. (2024). *Python's sorted() and sort() — A Primer*. <https://realpython.com/python-sort/>
- [8] Real Python. (2024). *Reading and Writing CSV Files in Python*. <https://realpython.com/python-csv/>